HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

OFFICE FOR INTERNATIONAL STUDY PROGRAMS

# DISCRETE STRUCTURE FOR COMPUTING (CO1007)

## Assignment 243

## TRAVELLING SALESMAN PROBLEM (TSP)

## TSP Report

**Instructors:** Tran Tuan Anh
Tran Hong Tai

**Class:** CC01

**Student:** Pham Quoc Vuong – 2353346

Ho Chi Minh – 2025

# Contents

**1. Background and Motivation ..... 1**

**2. Task Specification ..... 1**

**3. High-Level Method ..... 2**

**4. Input Normalization and Vertex Mapping ..... 2**

**5. Algorithm ..... 3**

5.1. Notation ..... 3

5.2. Candidate 1 — Hungarian + Subtour Patching ..... 3

5.3. Candidate 2 — Directed Greedy with Early-Cycle Avoidance ..... 3

5.4. Selecting the Initial Best Tour ..... 4

5.5. Local Search — Or-opt (k = 1, 2, 3) ..... 4

**6. Correctness (Pragmatic) ..... 4**

**7. Complexity ..... 5**

**8. Edge Cases ..... 5**

## 1. Background and Motivation

The Travelling Salesman Problem (TSP) asks for the shortest tour that visits each city exactly once and returns to the origin; it is NP-hard. TSP is commonly modeled as a Hamiltonian cycle on a weighted (typically complete) graph, and the number of tours grows factorially, making exact solutions computationally prohibitive on large instances.

Given strict time limits in the LMS code runner, the assignment **encourages heuristic/approximation algorithms** (e.g., Nearest Neighbor, Greedy, Christofides, 2-opt/3-opt) that offer a practical trade-off between solution quality and runtime.

## 2. Task Specification

Implement:

```
string Traveling(int graph[60][60], int n, char start)
```

which returns a string describing a tour that visits all vertices and returns to the starting vertex. The output format is the vertex names separated by a single whitespace, ending with the start vertex (e.g., “A B C … A”).

Input graphs are weight matrices of positive integers; **0 means “not connected.”** Vertices are named A..Z then a..z in order (A is index 0, B is index 1, …, a is index 26, …).

## 3. High-Level Method

The implementation is a **multi-start heuristic pipeline**:

- **Hungarian to build a low-cost 1-factor:** I solve a linear assignment on the directed cost matrix to obtain a permutation to[u] with exactly one outgoing and one incoming arc for each vertex, which typically decomposes into **a set of disjoint subtours** rather than a single Hamiltonian cycle.
- **Subtour patching (cycle merging):** I iteratively merge distinct cycles by **swapping two arcs across two cycles** whenever the swap is feasible and reduces total cost, aiming to produce a single tour.
- **Directed Greedy candidate:** Independently, I also construct a tour by sorting all feasible directed edges by weight and choosing edges while enforcing at most 1 in-degree and 1 out-degree per vertex, **avoiding early cycles** until the final edge closes the tour.
- **Local search (Or-opt with k = 1,2,3):** I refine the best candidate by relocating contiguous blocks of length k to different positions **iff** the boundary cost decreases. A simple step-budget cap = n*n*10 bounds runtime.

This design is aligned with the assignment's emphasis on heuristics under time limits.

## 4. Input Normalization and Vertex Mapping

I set INF = 1e9. For all i == j or graph[i][j] == 0, I treat the edge as **non-existent** and store INF in the working matrix w. (Matches the handout's "0 means not connected.")

Vertex names map as 0..25 ↔ 'A'..'Z', 26..51 ↔ 'a'..'z'. The start character is validated and converted to its index; invalid start or out-of-range n leads to returning the empty string.

# 5. Algorithm

## 5.1. Notation

- w[i][j]: normalized cost matrix (INF for missing/self edges).
- s: start index.
- cand: list of candidate tours (each a permutation of {0..n-1}).

## 5.2. Candidate 1 — Hungarian + Subtour Patching

(i) Hungarian assignment: Create a[1..n][1..n] by mapping w into finite costs; missing edges get a large finite BIG so the algorithm can run but selections of such edges mark infeasibility. Run the standard Hungarian method to obtain an assignment p[j], which I convert into a permutation to[i] on 0..n-1.

(ii) Patching/merging cycles. Decompose to into cycles C1, C2, …. While multiple cycles remain, search for a pair of arcs (i→ai) in C1 and (j→aj) in C2 such that the cross-swap (i→aj), (j→ai) is feasible and minimizes

$$\boldsymbol{\Delta = w[i][aj] + w[j][ai] - w[i][ai] - w[j][aj]}$$

Perform the best swap if found, thereby merging two cycles. Stop when there is a single cycle or no beneficial merge exists.

(iii) Read the tour: Follow to starting at s to produce an order of length n. The tour is **accepted only if** it is a **valid permutation** (all vertices appear exactly once) and its cycle cost is finite; otherwise it is discarded.

## 5.3. Candidate 2 — Directed Greedy with Early-Cycle Avoidance

Construct the edge list E = {(u,v,w[u][v]) | u≠v, w[u][v]<INF} and sort by increasing w.

Maintain degree constraints out[u] and in[v] (each ∈ {−1 or assigned}).

Iterate over E and add edge (u→v) if:

- out[u] == −1 and in[v] == −1, and
- inserting it does **not** create a directed cycle unless it is the **last** edge needed to close the tour.

If all out[u] are assigned, follow out from s to obtain a candidate tour; validate as a permutation with finite cycle cost.

### 5.4. Selecting the Initial Best Tour

Compute cost(tour) = Σ w[t[i]][t[i+1]] + w[t[n-1]][t[0]]. Choose the lower-cost tour among all valid candidates in cand. If cand is empty, return the empty string (no Hamiltonian tour exists under the given connectivity).

### 5.5. Local Search — Or-opt (k = 1, 2, 3)

For each block length k and each contiguous block [i, …, i+k-1], consider reinserting that block immediately **before** position j (with j not inside the block). Use a constant-time boundary evaluation:

- removal delta

$$\boldsymbol{rem = w[pre][blkL] + w[blkR][suf] - w[pre][suf]}$$

- insertion delta

$$\boldsymbol{add = w[a][blkL] + w[blkR][b] - w[a][b]}$$

  If add < rem, apply the move and restart the search. The loop halts when no improving move exists or when the step budget cap = n*n*10 is exhausted.

## 6. Correctness (Pragmatic)

- **Feasibility checks** ensure no missing edges are used; any candidate containing INF edges is rejected.
- **Permutation validation** prevents accepting a repeated-vertex cycle (a subtour).
- If the graph lacks a Hamiltonian cycle (per the input connectivity), the function returns the **empty string** by design.

This pragmatic notion of correctness matches the assignment's focus on producing a good tour quickly rather than proving optimality.

## 7. Complexity

- Hungarian: $\boldsymbol{O(n^3)}$.
- Directed Greedy: sorting $\boldsymbol{O(m \log m)}$ for $\boldsymbol{m \leq n(n-1)}$, then linear selection.
- Or-opt ($k \leq 3$): empirical $\boldsymbol{O(n^3)}$ with a small constant; the fixed step budget cap further bounds runtime.

Given $\boldsymbol{n \leq 52}$, these bounds are consistent with the code-runner time constraints described in the assignment.

## 8. Edge Cases

- If no Hamiltonian tour exists under the given connectivity (i.e., due to missing edges), the function intentionally returns the empty string.
- Disconnected instances (many zeros) may yield no Hamiltonian tour; the function then returns empty.
- Self-loops and 0 weights are ignored by normalization (INF).

End.